Vacation Finder

By Dinidu Samaranayake and Oliver Hankins

Functionality

The tool will be used to discover and find good vacation spots through a search engine. The user will input what they're looking for (ex: "cheap beach spot near Houston during summer") and it will go through the database of resorts, activities, etc. and return results with relevant information such as cost, location, places nearby, etc. It will do this by going through reviews and connecting with Google Maps in order to determine quality and relevance.

Why is this tool needed? Does it already exist?

Typically searches require you to have a specific location in mind, and even then it's hard to sort through multiple different sources that people have to cross-review in order to determine a good spot. Through this application, users won't have to do that and instead will have a tailored way to search and find what they want by fetching and considering location distance, cost parameters, and review sentiment into a single queryable search interface.

Platforms such as Google Maps, Expedia, and TripAdvisor offer search features but rely on specific destination names and have a strict lookup that isn't meant to handle long natural language queries.Instead of requiring rigid metadata filters, our tool searches across aggregated review texts, destination descriptions, and spatial proximity data simultaneously. It ranks destinations based on how well user reviews support the natural-language query. This will in turn cut planning time from hours down to a single search, making the process much more stress-free.

Plan

Data: We will likely use Kaggle datasets such as Inside Airbnb Open Data and the TripAdvisor Hotel & Resort Reviews for our primary document collection which will contain thousands of rich user reviews, listing descriptions, and amenity attributes. To satisfy the data-collection requirement and enrich our index, we'll write a custom Python script to fetch supplemental metadata from two public APIs: OpenStreetMap (Nominatim) for geocoding and relative driving distance calculations, and Open-Meteo for historical seasonal weather averages.

Core Algorithm:

- **Baseline:** Aggregate each destination's reviews and property descriptions into unified documents and rank results using the BM25 lexical scoring algorithm.
- **Atmospheric Detector:** Build a weighted lexicon of atmosphere terms (walkable, cozy, nightlife, family-friendly, uncrowded, etc.) and apply query expansion for atmospheric descriptors.

- **Semantic Matching:** Generate dense sentence embeddings using Hugging Face sentence transformers to compute cosine similarity between query phrases and unstructured review passages, allowing searches such as "quiet coastal retreat" to match reviews that express similar sentiment without sharing exact words.
- **Hybrid Ranker:** Combine BM25 lexical scores, dense vector similarity, and vibe-signal scores into a unified ranking function, adjusted by metadata multipliers for price caps, distance radius, and seasonal weather fit.

Application: A Python application backend which will likely be built with FastAPI or Streamlit that will serve as the retrieval index with a web frontend that allows users to enter natural language queries, adjust distance and price filters, and view ranked destination cards alongside supporting review snippets and location map views.

Existing Resources

Datasets & Public APIs: Inside Airbnb Open Data, TripAdvisor Hotel & Resort Reviews (Kaggle), OpenStreetMap (Nominatim API for geocoding), and Open-Meteo Historical & Climate API.

IR & Machine Learning Libraries: rank_bm25 (for baseline lexical indexing), sentence-transformers (for dense text embeddings), scikit-learn / FAISS (for similarity computations), pandas (for CSV data manipulation), and spaCy / NLTK (for text processing).

Web Framework & Hosting: Python with FastAPI or Streamlit for backend application logic, GitHub for code management, and Streamlit Community Cloud or Render for free public deployment.

How will usefulness be demonstrated

Offline Evaluation: We'll construct a benchmark set of 30–40 realistic, scenario-based travel queries covering various constraints (ex: "cheap beach spot near Houston under $150/night for summer" or "scenic mountain cabin with hiking trails") and then establish relevance judgments across the destination corpus. We'll then evaluate and compare the performance of the basic BM25 keyword search baseline against our hybrid ranker using standard IR metrics, specifically Precision@K, Mean Average Precision (MAP), and NDCG@10.

Ablation Study: We'll measure the incremental performance gains provided by each individual system layer to prove that our hybrid approach delivers superior search quality compared to basic keyword search.